# PROGRAMMAZIONE 2

a.a. 2023-2024

## allocazione dinamica della memoria

# Organizzazione della memoria assegnata ad un programma in esecuzione

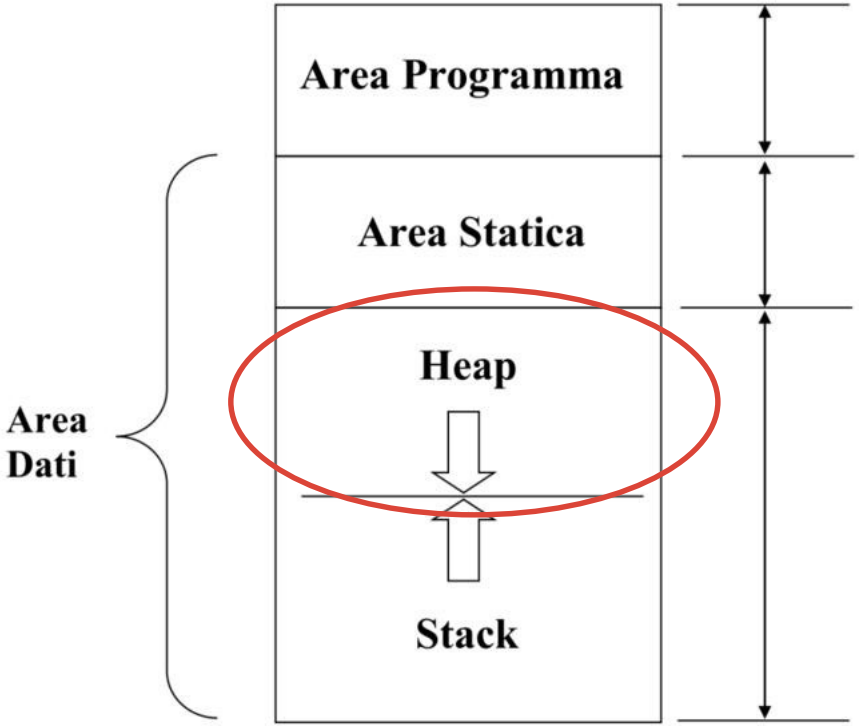
# Tipi di memoria dati in C

Nella modello di memoria della macchina astratta C esistono tre tipi di memoria per allocare le variabili:

1. **Memoria statica**, usata per memorizzare le variabili globali (e le variabili locali static).
2. **Memoria automatica (stack),** usata per memorizzare i record di attivazione delle funzioni. Ogni record di attivazione è costituito da:
   1. L'indirizzo di codice dell'istruzione successiva a quella che ha invocato la funzione (indirizzo di rientro);
   2. I parametri della funzione; e
   3. Le variabili locali (non static) alla funzione.
3. **Memoria dinamica (heap),** usata per allocare variabili (spazi di memoria) la cui dimensione si conosce solo in fase di esecuzione (Esempio: in un programma che dopo aver acquisito il numero di rilevazioni disponibili da una sonda di temperatura, le memorizzi in un vettore).[1]
   - In C, l'allocazione nella memoria heap e la sua liberazione sono espliciti, cioè si usano apposite chiamate a funzioni
     - La funzione malloc della libreria stdlib.h alloca uno spazio di memoria contiguo nell'heap e restituisce un puntatore al primo byte allocato. In caso di fallimento restituisce il valore NULL che corrisponde all'indirizzo riservato 0.
     - La funzione free della libreria stdlib.h libera lo spazio di memoria precedentemente allocato con la malloc.

[1] I VLA (variable-length array) sono stati introdotti in C99, con le versioni precedenti di C si deve usare la memoria dinamica per gestire questo esempio. Ma, come vedremo, la memoria dinamica ha anche altri utilizzi.

# Organizzazione della memoria assegnata ad un programma C in esecuzione




- Memorizza il programma (compilato e linkato) – *la sua dimensione è fissata a compile time.*
- Memorizza le variabili globali, dichiarate al di fuori delle funzioni, **allocate staticamente** (quando inizia l'esecuzione del programma) e accessibili da tutte le funzioni – *la sua dimensione è fissata a compile time.*
- Memorizza variabili **allocate dinamicamente** – *la sua dimensione varia durante l'esecuzione, ma la dimensione di STACK + HEAP è fissata a compile time.*
- Memorizza le variabili dichiarate all'interno delle funzioni che sono attive (**allocate automaticamente**, aggiungendo/togliendo un record di attivazione in cima allo stack, quando l'esecuzione di una funzione inizia/termina) – *la sua dimensione varia durante l'esecuzione.*

# Allocazione e deallocazione dinamiche

# Allocazione e deallocazioni dinamiche

Ovvero effettuate durante l’esecuzione del programma, occorre utilizzare delle funzioni di libreria:

```
#include <stdlib.h>
void *malloc(size_t size);
void free(void *ptr);
```

# Funzione malloc

Permette di allocare dinamicamente (durante l’esecuzione del programma) una quantità di memoria specificata.

---

**cast-type** ***ptr;**

…

ptr = (**cast-type** ***) malloc(**byte-size**);

# Funzione free

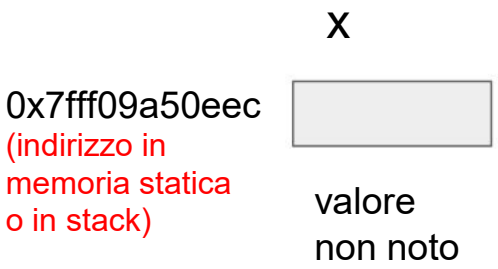
Permette di liberare dinamicamente (durante l’esecuzione del programma) della memoria precedentemente allocata in maniera dinamica. Dopo free tale memoria non sarà più considerata (dal run-time del C) come allocata – ovvero sarà (ri)allocabile.

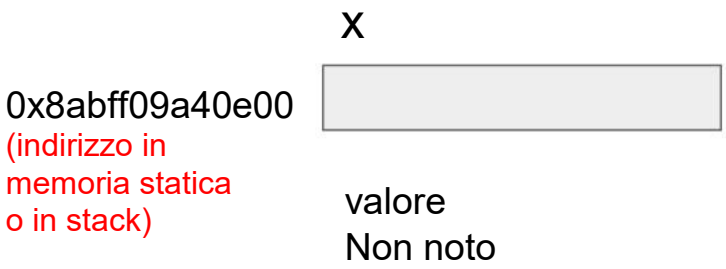
---

```
cast-type *ptr;

…

ptr = (cast-type *) malloc(byte-size);

free(ptr);
```

# Allocazione statica/automatica vs dinamica

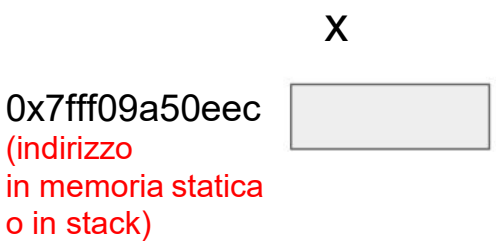
| STATICA o AUTOMATICA | DINAMICA |
| --- | --- |
| int x; // int | int *x; // indirizzo di un int (variabile statica o automatica) |

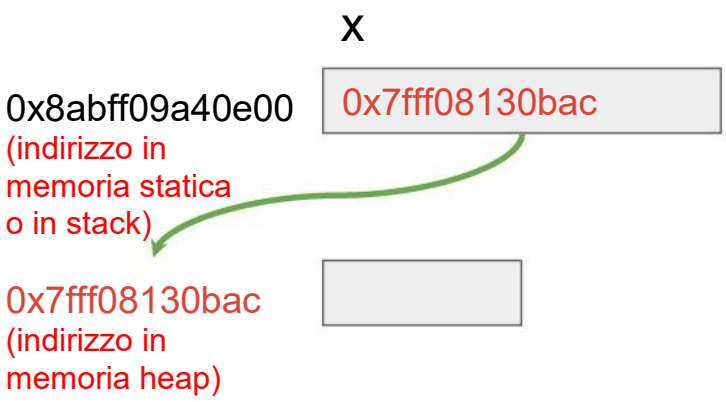


# Allocazione statica/automatica vs dinamica

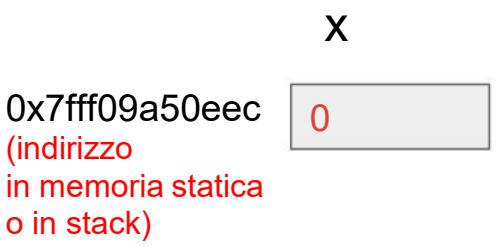
| STATICA o AUTOMATICA | DINAMICA |
| --- | --- |
| int x; | int *x;<br>x = (int *) malloc(sizeof(int)); |

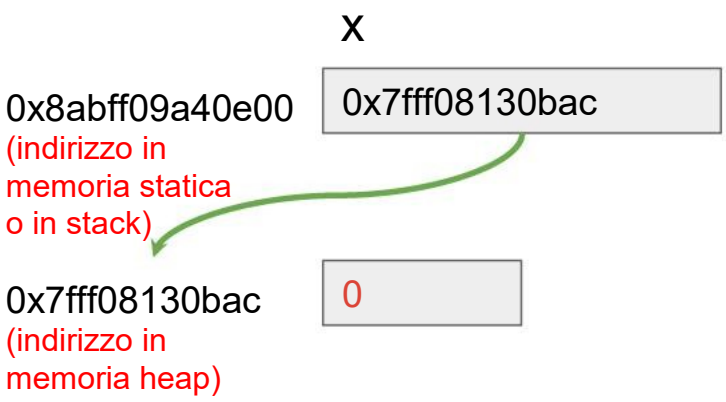


# Allocazione statica/automatica vs dinamica

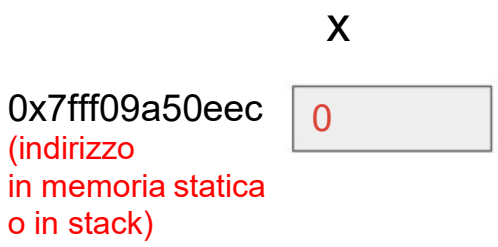
| STATICA o AUTOMATICA | DINAMICA |
| --- | --- |
| int x;<br>x = 0; | int *x;<br>x = (int *) malloc(sizeof(int));<br>*x = 0; |

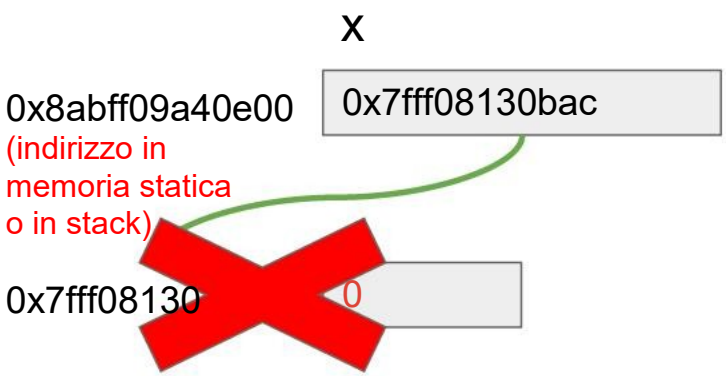


# Deallocazione: funzione free

| STATICA o AUTOMATICA | DINAMICA |
| --- | --- |
| int x;<br>x = 0; | int *x;<br>x = (int *) malloc(sizeof(int));<br>*x = 0;<br>**free(x);** |



# Confronto schematico

| STATICA/AUTOMATICA – tempo | DINAMICA – spazio |
| --- | --- |
| ➔ accesso più rapido<br>➔ occupo memoria anche per variabili che non uso | ➔ accesso meno rapido<br>➔ occupo solo la memoria che mi serve |

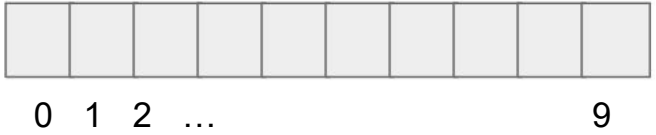
```
TIPO VRB;
if (COND) USO VRB;
else NON USO VRB
```

```
int ARRAY[100];
USO SOLO PRIME 5 CELLE
```

# Allocazione di array

# Array (Prog 1 – recap)

int v[10];




quanto spazio occupa?

–

posso scrivere v+1?

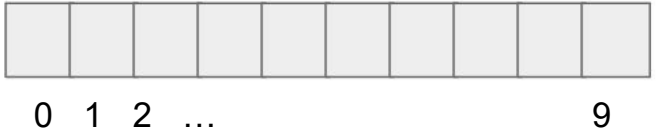
–

posso scrivere v++?

–

# Array

int v[10];




quanto spazio occupa?

**sizeof(v) == sizeof(int)*10 == 40 byte**

posso usare v+1?

**sì, es: printf(“%d”, *(v+1))  *(v+1) equivale a v[1]**

posso fare v++?

**no, v non può essere modificato**

# Allocazione dinamica di un array

```
int *d;

...

d = (int *)malloc(sizeof(int)*10);
```

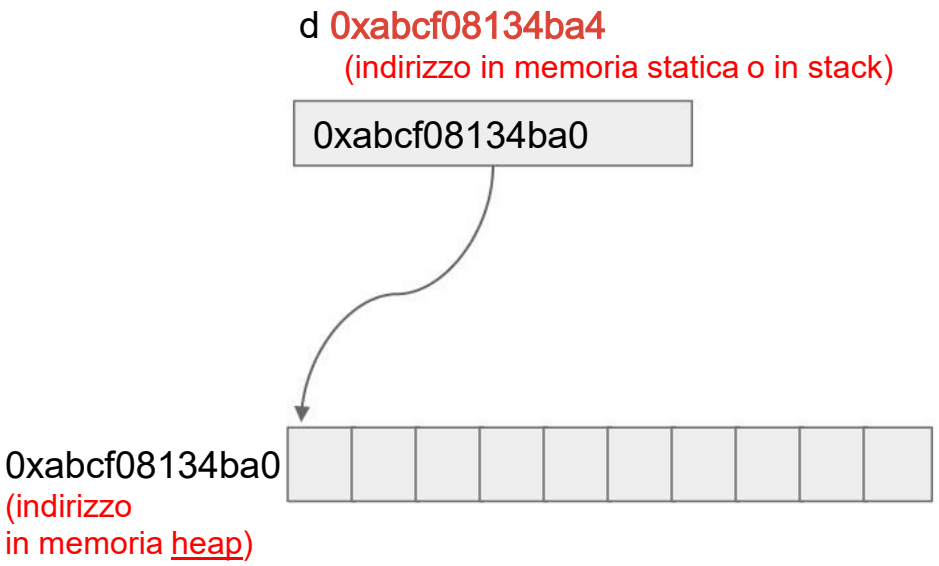
d 0xabcf08134ba4

(indirizzo in memoria statica o in stack)

val. non noto

# Allocazione dinamica di un array

```
int *d;

…

d = (int *)malloc(sizeof(int)*ESPR);
```

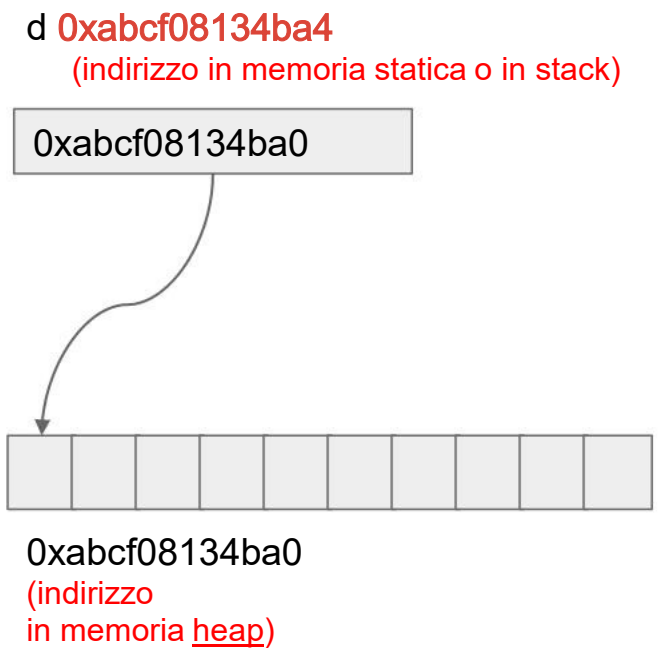


# Generalizziamo

Sia T un tipo qualsiasi:

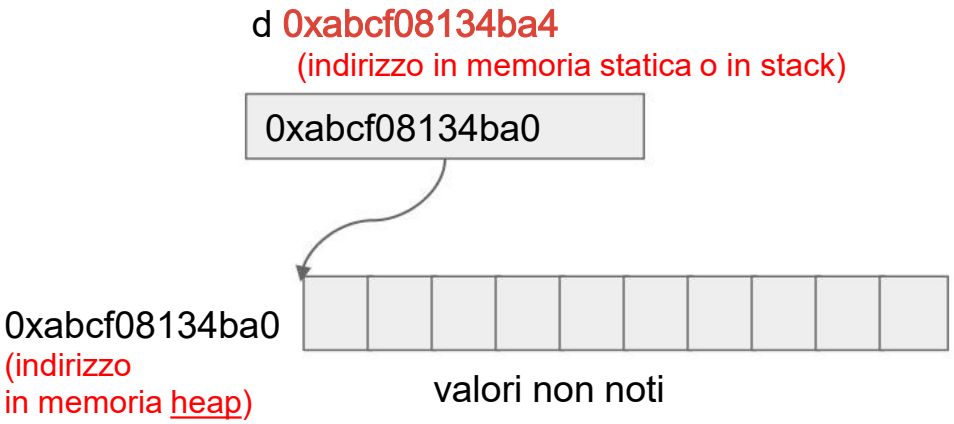
```
T *d;
...
d = (T *)malloc(sizeof(T)*ESPR);
```

Quanto detto su array di int può essere generalizzato ad array di un qualsiasi **tipo T**



# Allocazione dinamica di un array

```
T *d;

d = (T *)malloc(sizeof(T)*ESPR);
```




quanto spazio occupa?

–

posso usare d+1?

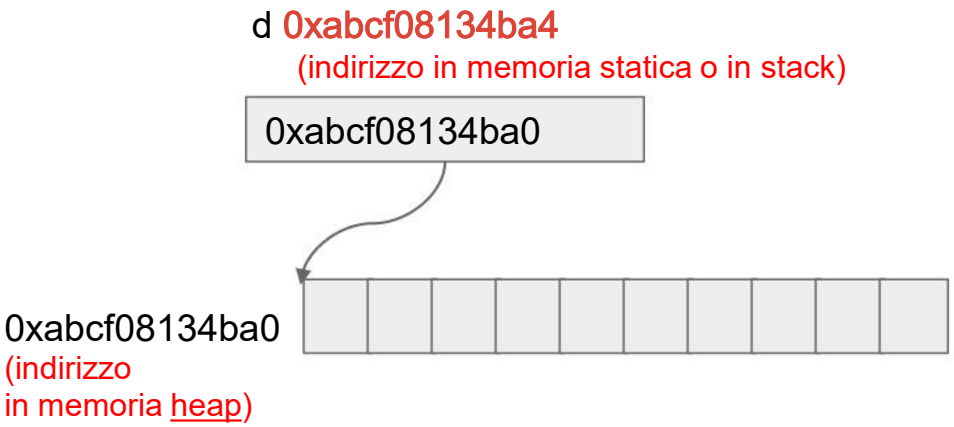
–

posso usare d++?

–

# Allocazione dinamica di un array

```
T *d;

d = (T *)malloc(sizeof(T)*ESPR);
```




quanto spazio occupa?

**spazio per l'array + sizeof(T *), cioè +8 byte**

posso usare d+1?

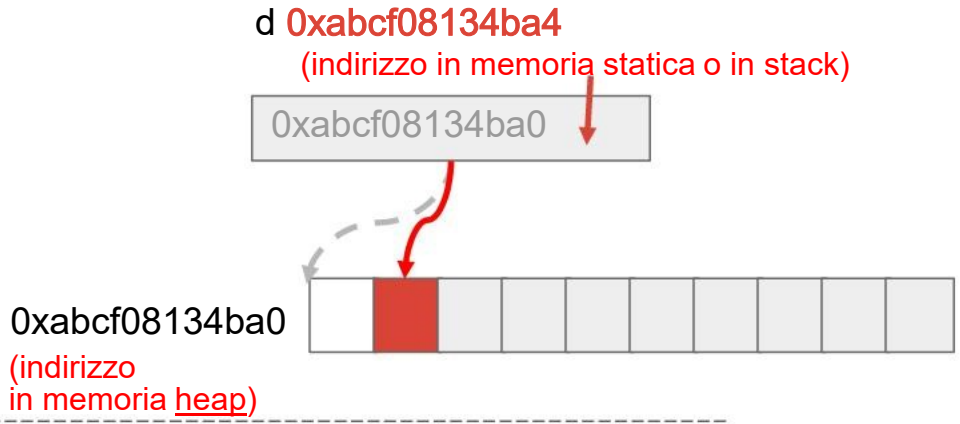
**sì**

posso usare d++?

**sì, vedi slide successiva**

# Effetto di d++

```
T *d;

d = (T *)malloc(sizeof(T)*ESPR);

d++;
```

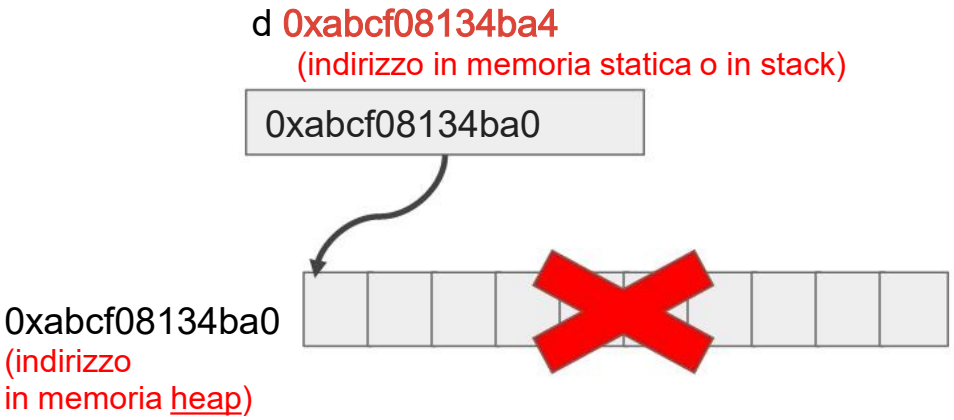



La variabile d ha un valore che viene incrementato del numero di byte occupati da un elemento dell'array e punterà al secondo elemento

Es. nel caso di elementi interi viene incrementata di 4. L'elemento precedentemente in posizione 0 è accessibile attraverso l'espressione d-1

# Deallocazione di un array

```
T *d;

d = (T *)malloc(sizeof(T)*ESPR);

free(d);
```

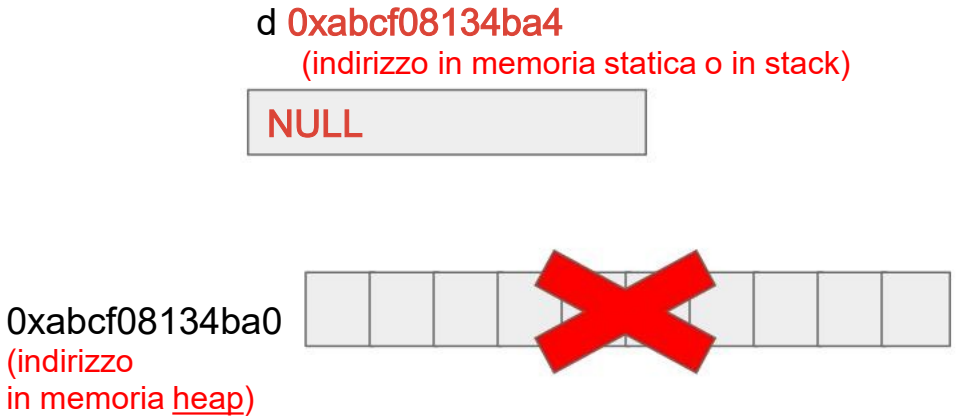



---

la porzione di RAM occupata dall'array sarà considerata (dal run-time del C) come non allocata -- e quindi (ri)allocabile.

# Deallocazione di un array

```
T *d;

d = (T *)malloc(sizeof(T)*ESPR);

free(d);

d = NULL;
```




---

la porzione di RAM occupata dall'array sarà considerata come non allocata dal run-time del C

**ATTENZIONE: il valore di d non diventa NULL per effetto della free!**

# Esempio di uso errato

```
char *d = (char *) malloc(sizeof(char)*10);

…

free(d);   // senza mettere a NULL il puntatore

…

if (d != NULL) {

        printf(…, d[0]); …

…
```

il valore NULL del puntatore segnala che non si riferisce ad un'area di memoria allocata. Occorre mantenere la **consistenza** se no il suo valore sarà non significativo e l'esecuzione successiva sarà **errata.**

# Attenzione!

È possibile che per un po' lo spazio liberato con free sembri ancora disponibile all'esecuzione: gli assegnamenti/le printf non danno errore

Tuttavia quella memoria può essere riassegnata per un altro uso **in qualsiasi momento**!

Mettereste qualcosa di utile in una cassettiera in strada, dopo avere chiamato l'azienda per lo smaltimento dei rifiuti?

# Passaggio di array come parametro

```
int main() {
    int astatico[N];
    int *adinamico=(int *)malloc(sizeof(int)*N);

    init(astatico);
    stampa(astatico);

    init(adinamico);
    stampa(adinamico);
}
```

L’effetto sarà uguale o diverso?

# Passaggio di array come parametro

```
int main() {
    int astatico[N];
    int *adinamico=(int *)malloc(sizeof(int)*N);

    init(astatico);
    stampa(astatico);

    init(adinamico);
    stampa(adinamico);
}
```

Sarà uguale perché anche il nome di un array statico corrisponde a un puntatore

# Allocazione di struct

# Un esempio di strut

```
typedef struct {
        char titolo[MAXT];
        int pagine;
        char autore[MAXN];
        float prezzo;
    } Libro ;

Libro L1;
```

QUESTO È UN PROTOTIPO DI STRUTTURA

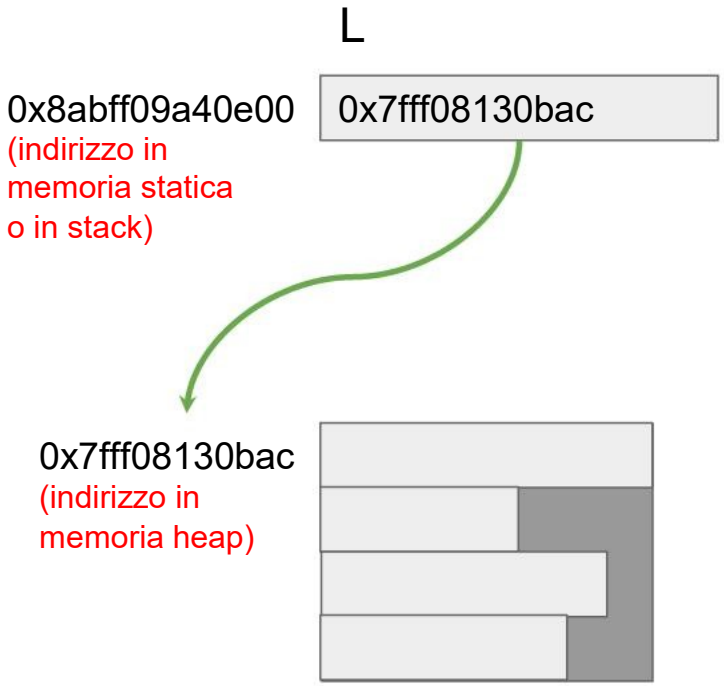
NON HA ALLOCATA MEMORIA

OCCORRE DICHIARARE VARIABILI DI TIPO “Libro”

# Strutture allocate dinamicamente

Libro * L;

L = (Libro *) malloc(sizeof(Libro);

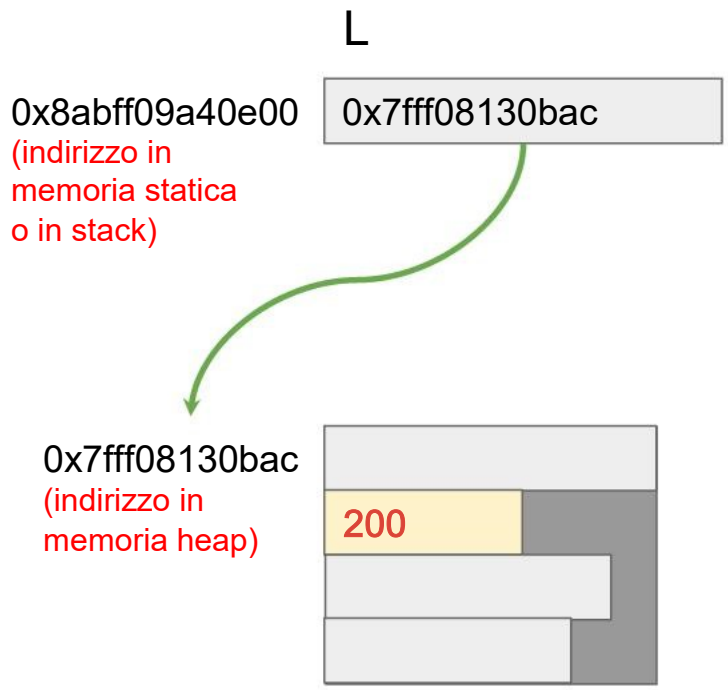


# Strutture allocate dinamicamente – accesso ai campi

```
Libro * L;

L = (Libro *) malloc(sizeof(libro);

(*L).pagine = 200;
```

oppure più solitamente

```
L -> pagine = 200;
```



# Strutture allocate dinamicamente – free

Libro * l;

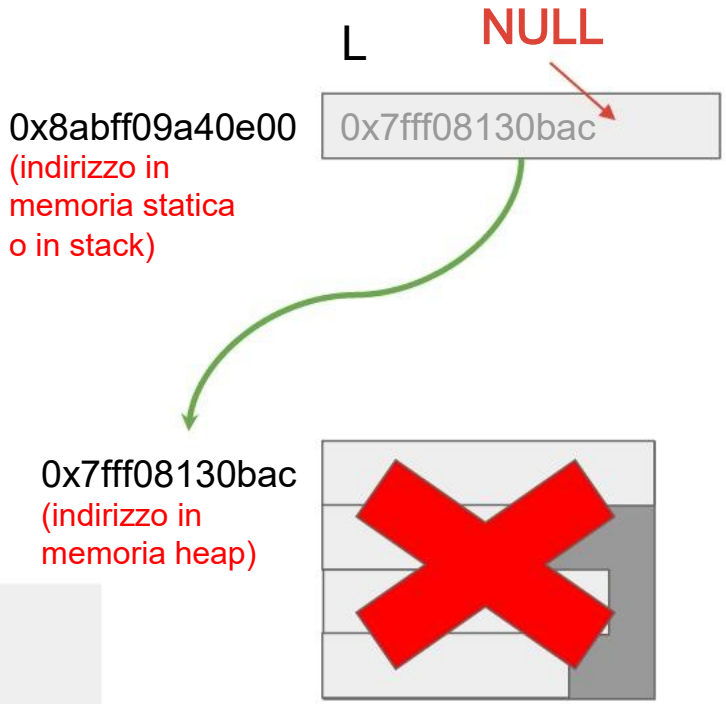
l = (Libro *) malloc(sizeof(libro);

…

**free(l);**

**l = NULL;**

Necessarie entrambe per mantenere la consistenza della implementazione



# Passaggio di struct come parametro

```
struct prova {
   int x;
   int y;
};

void stampa(struct prova p) {
    printf("\n x=%d\n", p.x);
    printf(" y=%d\n", p.y);
}
```

# Passaggio di struct come parametro

```
int main() {
    struct prova statica;
    struct prova *dinamica = (struct prova *) malloc(sizeof(struct prova));
    // inizializzo statica a 10, 10 e dinamica a 20, 20
    statica.x = 10; statica.y = 20;
    dinamica->x = 20; dinamica->y = 20;
```

# Passaggio di struct come parametro

come invoco stampa sulle due variabili?

statica

dinamica

stampa(**statica**)

stampa(**&statica**)

stampa(**dinamica**)

stampa(***dinamica**)

?

# Passaggio di struct come parametro

come invoco stampa sulle due variabili?

statica

stampa(**statica**)

A differenza degli array solo "dinamica" è un puntatore

dinamica

stampa(***dinamica**)

# Passaggio di struct come parametro

```
struct prova {
    int x;
    int y;
};

void init(struct prova *p) {
    p->x = 15;
    p->y = 25;
}
```

init è una funzione che modifica la variabile passata come argomento. A questo fine dobbiamo necessariamente usare un puntatore

# Passaggio di struct come parametro

come invoco init sulle due variabili?

statica

dinamica

init(statica)

init(&statica)

init(dinamica)

init(&dinamica)

?

# Passaggio di struct come parametro

## come invoco init sulle due variabili?

statica

dinamica

per far modificare statica occorre passare il suo indirizzo (puntatore a statica)

init(**&statica**)

init(**dinamica**)

invece dinamica è già l'indirizzo della struct allocata dinamicamente

# Complementi sul C: funzioni variadiche

# Il prototype di `printf`

```
int printf(const char *format, ...);
```

- I puntini di sospensione (...) nel prototipo della funzione indicano un numero variabile di argomenti di qualsiasi tipo.

# Il prototype di `printf`

```
int printf(const char *format, ...);
```

- I puntini di sospensione (...) nel prototipo della funzione indicano un numero variabile di argomenti di qualsiasi tipo.
- Si può utilizzare la sintassi "$\mathtt{T}_0$ `f(`$\mathtt{T}_1$ $\mathtt{x}_1$`,`$\cdots$`,`$\mathtt{T}_n$ $\mathtt{x}_n$`,...);`" dove $n \geq 1$ per dichiarare una funzione con elenchi di argomenti a lunghezza variabile:
  - la funzione deve avere almeno un parametro diverso dai puntini di sospensione; e
  - i puntini di sospensione possono essere usati solo come ultimo parametro.

# Come scrivere funzioni variadiche in C

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$\texttt{T}_0$ `f(`$\texttt{T}_1$ $\texttt{x}_1$`,`$\cdots$`,`$\texttt{T}_n$ $\texttt{x}_n$`,...);`"?

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$\texttt{T}_0$ `f(`$\texttt{T}_1$ $\texttt{x}_1$`,`$\cdots$`,`$\texttt{T}_n$ $\texttt{x}_n$`,...);`"?

- Si deve includere `<stdarg.h>` e si devono usare in modo opportuno (nel body della funzione) `va_list`, `va_start`, `ca_arg`, e `va_end`. Ovvero:

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$\mathtt{T}_0$ `f(`$\mathtt{T}_1$ $\mathtt{x}_1$`,`$\cdots$`,`$\mathtt{T}_n$ $\mathtt{x}_n$`,...);`"?

- Si deve includere `<stdarg.h>` e si devono usare in modo opportuno (nel body della funzione) `va_list`, `va_start`, `ca_arg`, e `va_end`. Ovvero:
  1. Per accedere all'elenco di argomenti a lunghezza variabile, si deve dichiarare una variabile – chiamiamola `ap` – di tipo `va_list`
     - conterrà le informazioni necessarie alle macro `va_start`, `ca_arg`, e `va_end`.

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$T_0$ `f(`$T_1$ $x_1$`,`$\cdots$`,`$T_n$ $x_n$`,...);`"?

- Si deve includere `<stdarg.h>` e si devono usare in modo opportuno (nel body della funzione) `va_list`, `va_start`, `ca_arg`, e `va_end`. Ovvero:
  1. Per accedere all'elenco di argomenti a lunghezza variabile, si deve dichiarare una variabile – chiamiamola `ap` – di tipo `va_list`
     - conterrà le informazioni necessarie alle macro `va_start`, `ca_arg`, e `va_end`.
  2. Prima di accedere all'elenco di argomenti a lunghezza variabile, si deve invocare la macro `va_start(ap,` $x_n$ (dove $x_n$ è l'ultimo parametro fisso della funzione)
     - inizializza la variabile di tipo `va_list`

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$\mathtt{T}_0$ `f(`$\mathtt{T}_1$ $\mathtt{x}_1$`,`$\cdots$`,`$\mathtt{T}_n$ $\mathtt{x}_n$`,...);`"?

- Si deve includere `<stdarg.h>` e si devono usare in modo opportuno (nel body della funzione) `va_list`, `va_start`, `ca_arg`, e `va_end`. Ovvero:
  1. Per accedere all'elenco di argomenti a lunghezza variabile, si deve dichiarare una variabile – chiamiamola `ap` – di tipo `va_list`
     - conterrà le informazioni necessarie alle macro `va_start`, `ca_arg`, e `va_end`.
  2. Prima di accedere all'elenco di argomenti a lunghezza variabile, si deve invocare la macro `va_start(ap,` $\mathtt{x}_n$ (dove $\mathtt{x}_n$ è l'ultimo parametro fisso della funzione)
     - inizializza la variabile di tipo `va_list`
  3. Per accedere – nell'ordine – a ciascun elemento della lista degli $m \geq 0$ argomenti a numero variabile, si deve invocare per ciascun argomento $\mathtt{arg}_{n+1}, \cdots, \mathtt{arg}_{n+m}$ la macro `va_arg(ap,` $\mathtt{T}_{n+i}$ (dove $\mathtt{T}_{n+i}$, con $1 \leq i \leq m$, è il tipo di $\mathtt{arg}_{n+i}$)
     - resituisce il valore dal prossimo argomento nella lista degli $m \geq 0$ argomenti a numero variabile.

# Funzioni variadiche

Sono funzioni che accettano un numero variabile di argomenti di qualsiasi tipo. Come fare per scrivere il body una funzione con prototype "$\mathtt{T}_0$ `f(`$\mathtt{T}_1$ $\mathtt{x}_1$`,`$\cdots$`,`$\mathtt{T}_n$ $\mathtt{x}_n$`,...);`"?

- Si deve includere `<stdarg.h>` e si devono usare in modo opportuno (nel body della funzione) `va_list`, `va_start`, `ca_arg`, e `va_end`. Ovvero:
  1. Per accedere all'elenco di argomenti a lunghezza variabile, si deve dichiarare una variabile – chiamiamola `ap` – di tipo `va_list`
     - conterrà le informazioni necessarie alle macro `va_start`, `ca_arg`, e `va_end`.
  2. Prima di accedere all'elenco di argomenti a lunghezza variabile, si deve invocare la macro `va_start(ap,` $\mathtt{x}_n$ (dove $\mathtt{x}_n$ è l'ultimo parametro fisso della funzione)
     - inizializza la variabile di tipo `va_list`
  3. Per accedere – nell'ordine – a ciascun elemento della lista degli $m \geq 0$ argomenti a numero variabile, si deve invocare per ciascun argomento $\mathtt{arg}_{n+1}, \cdots, \mathtt{arg}_{n+m}$ la macro `va_arg(ap,` $\mathtt{T}_{n+i}$ (dove $\mathtt{T}_{n+i}$, con $1 \leq i \leq m$, è il tipo di $\mathtt{arg}_{n+i}$)
     - resituisce il valore dal prossimo argomento nella lista degli $m \geq 0$ argomenti a numero variabile.
  4. Quando si è finto di accedere accedere all'elenco di argomenti a lunghezza variabile, si deve invocare la macro `va_end(ap`
     - Se `va_end` non viene chiamata prima di ritornare dalla funzione, il risultato non sarà definito.

# Un esempio: calcolo della media di $n \geq 1$ interi

```c
#include <stdio.h>

double average(int i, ...); // ... represents variable arguments

int main(void) {
    double w = 37.5;
    double x = 22.5;
    double y = 1.7;
    double z = 10.2;

    printf("%s%.1f; %s%.1f; %s%.1f; %s%.1f\n\n",
        "w = ", w, "x = ", x, "y = ", y, "z = ", z);
    printf("%s%.3f\n%s%.3f\n%s%.3f\n",
        "The average of w and x is ", average(2, w, x),
        "The average of w, x, and y is ", average(3, w, x, y),
        "The average of w, x, y, and z is ", average(4, w, x, y, z));
}

// calculate average
double average(int i, ...) {
    double total = 0; // initialize total
    va_list ap; // stores information needed by va_start and va_end

    va_start(ap, i); // initializes the va_list object

    // process variable-length argument list
    for (int j = 1; j <= i; ++j) {
        total += va_arg(ap, double);
    }

    va_end(ap); // clean up variable-length argument list
    return total / i; // calculate average
}
```

# Due domande

1. Come mai il codice nel lucidi precedente non include `<stdarg.h>`?

# Due domande

1. Come mai il codice nel lucidi precedente non include `<stdarg.h>`?
2. Come fa `printf` a a sapere quale tipo usare per ogni chiamata ala macro `va_arg.h`?

# Complementi sul C: Variabili locali static

# Variabili locali static

Una variabile locale (cioè dichiarata all'interno di una funzione) static è una variabile che mantiene il suo valore dopo che la funzione è terminata.

Ad esempio:

```
int nextNumber() {
    static int counter = 1;
    return counter++;
}
```

restituisce:

- 1 la prima volta che è chiamata
- 2 la seconda volta che è chiamata
- …
- n (a meno di overflow) l'n-esima volta che è chiamata

# Complementi sul C:
# Variabili globali e qualificatori
# extern e static

# Variabili globali e qualificatori extern e static

1. Una variabile globale è una variabile dichiarata al top level.
   - Se ``T v'' è una varabile globale dichiarata in un file nomeFile.c, allora:
     - è accessibile da tutte le funzioni definite (dopo) in nomeFile.c
     - è accessibile, in altri file che contengono la dichiarazione ``extern T v;'', in tutte le funzioni definite dopo tale dichiarazione
     - in nomeFile.c, una dichiarazione ``extern T v;'' può precedere la definizione ```T v'' -- infatti **una dichiarazione ``extern T v;'' significa che: la variabile ``T v'' è dichiarata in seguito nello stesso file oppure è dichiarata in un altro file**.
   - Se ``static T v'' è una varabile globale dichiarata in un file nomeFile.c, allora:
     - è accessibile da tutte le funzioni definite (dopo) in nomeFile.c
     - NON è accessibile in altri file

2. La visibilità dei prototype delle funzioni è gestita in modo analogo
   - Nota: per i prototype di funzioni il qualificatore extern è implicito, quindi non serve scriverlo.

**Per il genere di programmi che incontreremo a Prog. II non dovrebbero servire variabili globali.**